

Construir la app con Spec Kit

AI Media Generator — 39 tickets en 6 fases, listos para ejecutar.

Repo github.com/sjunka/speckit-ai-generator

Feature 001-ai-media-generator

Fecha 2026-08-18

Qué vamos a hacer

Una app donde el usuario toma una foto, la IA la convierte en una imagen bonita, y esa imagen se vuelve un video corto que puede descargar o compartir.

No vamos a escribir el código a mano. Ya está todo descrito en este repo: qué hace la app, cómo se construye, y en qué orden. El agente de IA lee esa descripción y escribe el código.

Tu trabajo es pasarle los tickets uno por uno y revisar que salga bien.

Qué necesitas

- Claude Code instalado
- Una API key de Anthropic que funcione
- Node 20 o superior
- git

Nada más. No hace falta instalar Spec Kit, ya viene en el repo.

Lista de tareas

- ☐ 1. Clonar el repo — todos
- ☐ 2. Verificar que funciona — todos
- ☐ 3. Leer tres documentos — todos
- ☐ 4. Ticket T001, crear el proyecto — Sergio
- ☐ 5. Tickets T002 a T009, la base — Sergio
- ☐ 6. Repartir: T010 Mateo, T018 Johan, T026 Tomás
- ☐ 7. Tickets T031 a T039, juntar todo — Sergio
- ☐ 8. Desplegar — Sergio

Son 39 tickets en total. Van del T001 al T039, en orden.

Los pasos 4 y 5 hay que hacerlos **antes** de la presentación: bloquean a todos los demás y tardan más de una hora.

1. Clonar el repo

```
git clone https://github.com/sjunka/speckit-ai-generator.git
cd speckit-ai-generator
```

2. Verificar que funciona

Antes de nada, comprueba que el agente encuentra los archivos:

```
bash .specify/scripts/bash/check-prerequisites.sh --json --require-tasks --include-tasks
```

Si te sale una línea larga que dice `FEATURE_DIR`, todo bien, sigue.

Si dice `Feature directory not found`, falta un archivo. Créalo así y vuelve a probar:

```
printf '{\n  "feature_directory": "specs/001-ai-media-generator"\n}\n' > .specify/feature.json
```

3. Leer tres documentos

Media hora de lectura que ahorra días de trabajo.

1. `docs/PROMPT-NOTES.md` — el más corto. Explica por qué el trabajo está partido en 6 fases y no de otra forma.
2. `specs/001-ai-media-generator/spec.md` — qué hace la app, sin tecnología.
3. `docs/REPLICATION-PROMPT.md`, **sección §10** — 16 errores comunes. Cada uno costó una hora la primera vez. Leerlos es la mejor media hora del proyecto.

El archivo `docs/REPLICATION-APPENDIX.md` **no se lee**. Es la respuesta correcta: tiene los 80 archivos del proyecto original. Sirve para comparar cuando dudas de algo.

4. Primer ticket

```
/speckit-implement T001
```

Cuatro cosas que se confunden mucho:

- El comando es `/speckit-implement`, **no** `/implement`. Spec Kit le pone el prefijo `speckit-` a todos sus comandos para no chocar con otras herramientas.
- El ticket es `T001`, **no** "ticket 1". Con la T y con tres dígitos, igual que está escrito en `tasks.md`.

- Escribir un solo ticket corre **solo ese ticket**. Para hacer un bloque entero, se le dice la fase: `/speckit-implement Phase 3 only (T010 to T017)`.
- **Nunca lo corras sin argumento.** `/speckit-implement` a secas construye los 39 tickets de golpe, incluidos los de tus compañeros.

Esto crea el proyecto de Next.js desde cero. Espera a que termine.

Cómo saber que salió bien: deben existir `package.json`, `app/layout.jsx` y `app/page.jsx`. Dentro de `package.json` debe decir `next: 16.3.x` y `react: 19.2.x`.

Este ticket ya se probó y funciona: crea Next.js 16.3.1 con React 19.2.8 y Tailwind 4, que es justo lo que pide el plan.

5. La base (T002 a T009)

Ocho tickets más. En vez de escribirlos uno por uno, se le pide la fase completa:

```
/speckit-implement Phase 2 only (T002 to T009)
```

Aquí se construyen los colores, la tipografía, los botones, y la configuración de las pruebas.

Si se atora en alguno, córrelo suelto para aislar el problema: `/speckit-implement T005`.

Cómo saber que salió bien:

```
npm run lint && npm test && npm run build
```

Los tres tienen que pasar. Cuando pasen, sube esto a `main`.

Nadie puede empezar el siguiente paso hasta que esto esté en `main`.

6. Repartir el trabajo

Aquí es donde tres personas trabajan al mismo tiempo sin estorbarse.

Persona	Rama	Tickets	Qué construye
Mateo	<code>001-frontend</code>	T010 a T017	Las pantallas que ve el usuario
Johan	<code>001-backend</code>	T018 a T025	El backend y la conexión con la IA
Tomás	<code>001-dashboard</code>	T026 a T030	El dashboard del dueño y las cuentas

Cada uno, en su propio computador:

```
git pull origin main
git checkout -b 001-frontend
/speckit-implement Phase 3 only (T010 to T017)
```

Johan usa `001-backend` y `Phase 4 only (T018 to T025)`. Tomás usa `001-dashboard` y `Phase 5 only (T026 to T030)`.

¿Por qué no chocan? Porque cada grupo toca archivos completamente distintos. En `tasks.md` está escrito qué archivos son de cada quien y cuáles no puede tocar. Ningún archivo aparece en dos listas.

Un aviso sobre los tickets de Tomás: el T030 es el único que necesita cuentas reales (Clerk, MongoDB, Vercel, Higgsfield) y el archivo `.env.local`. Sergio se lo tiene que pasar aparte, nunca por el repo.

Si trabajas solo: haz los tickets en orden, del T010 al T030, sin ramas. Funciona igual, solo que más lento.

Los comandos exactos de cada persona, momento a momento, están más abajo en **Cómo lo hacemos entre todos**.

7. Juntar todo (T031 a T039)

```
/speckit-implement Phase 6 only (T031 to T039)
```

Aquí se juntan las tres ramas y se arregla lo que se rompió.

El ticket importante de esta parte es el T032. Mientras trabajaban por separado, la persona A probó su código contra un backend falso. Ahora hay que comprobar que el backend de verdad se comporta igual. Si no coinciden, las pruebas pasan pero la app no funciona. No te saltes ese ticket.

8. Desplegar

El T039 lo explica. Vercel se conecta al repo y despliega solo. Solo hay que poner las 8 variables de entorno en la configuración del proyecto.

Cómo lo hacemos entre todos (4 personas)

Somos cuatro y cada uno tiene su bloque. Esta es la parte que hay que ensayar.

Quién	Rama	Tickets
Sergio	main	T001 a T009, después T031 a T039
Mateo	001-frontend	T010 a T017
Johan	001-backend	T018 a T025
Tomás	001-dashboard	T026 a T030

Antes de la presentación

Esto NO se hace en vivo. Los tickets T001 a T009 bloquean a todos y tardan un rato largo. Si los corres en la presentación, los otros tres se quedan mirando media hora.

Sergio, el día anterior:

```
git clone https://github.com/sjunka/speckit-ai-generator.git
cd speckit-ai-generator

/speckit-implement Phase 1 and Phase 2 only (T001 to T009)
```

Son nueve tickets. El agente los hace seguidos y marca cada uno cuando termina. Si se atora en alguno, córrelo suelto para aislarlo: `/speckit-implement T005`.

Cuando acaben los nueve:

```
npm run lint && npm test && npm run build
git add -A
git commit -m "Base del proyecto (T001 a T009)"
git push origin main
```

Los tres comandos tienen que pasar antes de subir. Si alguno falla, arréglalo antes de seguir — todo lo demás se construye encima de esto.

Sergio también: pásale el archivo `.env.local` a Tomás por WhatsApp o correo. **No lo subas al repo.** Tomás lo necesita para el T030, que es el único ticket que habla con las cuentas de verdad.

Los cuatro, la noche antes: clonar el repo y comprobar que al escribir `/speckit-` en Claude Code aparecen los comandos. Si no aparecen, estás fuera de la carpeta del repo.

Durante la presentación

Momento 1 — Sergio abre (2 minutos)

Enseña el repo. Explica la idea: nadie escribió el código, está descrito en `specs/` y el agente lo construye. Muestra `tasks.md` y los 39 tickets.

Momento 2 — los tres arrancan a la vez

Cada uno en su computador, al mismo tiempo:

```
# Mateo
git pull origin main
git checkout -b 001-frontend
/speckit-implement Phase 3 only (T010 to T017)
```

```
# Johan
git pull origin main
git checkout -b 001-backend
/speckit-implement Phase 4 only (T018 to T025)
```

```
# Tomás
git pull origin main
git checkout -b 001-dashboard
/speckit-implement Phase 5 only (T026 to T030)
```

Esta es la parte que vale la pena mostrar: tres agentes escribiendo código a la vez, en el mismo proyecto, sin chocar.

Momento 3 — Sergio explica mientras los agentes trabajan

Abre `tasks.md` y muestra los bloques que dicen *Owns* y *Never touches*. Ahí está la razón de que no choquen: ningún archivo aparece en dos listas.

Momento 4 — cada uno sube su rama

```
git add -A
git commit -m "Mi bloque de tickets"
git push -u origin 001-frontend      # o 001-backend, o 001-dashboard
```

Momento 5 — Sergio junta todo

```
git checkout main
git pull origin main
git merge 001-frontend 001-backend 001-dashboard
```

Si hay conflictos, van a ser en `package.json` y se arreglan quedándose con las dos listas de dependencias. Después:

```
/speckit-implement T032
npm test
```

El T032 es el ticket que salva la demo. Mateo probó su pantalla contra un backend falso, no contra el de Johan. El T032 comprueba que los dos coinciden. Si te lo saltas, las pruebas pasan y la app no funciona.

Momento 6 — la app corriendo de verdad

```
npm run dev
```

Con el `.env.local` puesto. Abre el navegador y haz el recorrido completo: entrar, foto, imagen generada, video, descargar. Ese es el cierre.

Cuántos tickets correr en vivo

Cada ticket tarda entre 5 y 15 minutos. Ocho tickets seguidos son más de una hora, y eso no cabe en una presentación.

Lo recomendable: construyan todo el día anterior, hasta tener la app funcionando. En la presentación, cada uno corre **solo su primer ticket** en vivo para enseñar cómo funciona, y el resto de su rama ya está hecho. Terminan con el merge y la app corriendo.

Así saben cuánto tarda cada cosa, y si algo falla lo descubren en su casa y no frente al profesor.

Si algo falla

Qué ves	Qué hacer
Feature directory not found	Falta <code>.specify/feature.json</code> . Ver el paso 2
El comando <code>/speckit-implement</code> no existe	Estás fuera de la carpeta del repo, o usas otro agente que no es Claude Code
El agente empieza a construir tickets que no son tuyos	Corriste <code>/speckit-implement</code> sin argumento. Detenlo, descarta los cambios, y vuelve a correrlo diciendo tu fase
El agente reescribe <code>tasks.md</code>	Alguien corrió <code>/speckit-tasks</code> . Reviértelo. Ese archivo no se regenera nunca
Un botón sale sin estilos	Error 16 de la sección §10
Una clase de Tailwind no hace nada	Error 15 de la sección §10. Tailwind no avisa cuando te equivocas de nombre
La imagen generada no aparece	Error 13 de la sección §10
El agente se inventa detalles	No leyó las secciones §6 y §7. Pásaselas a mano

Seis reglas que no se rompen

1. **La prueba primero.** Se escribe la prueba que falla, después el código que la arregla. En ese orden.
2. **Todo funciona sin internet.** Ninguna prueba necesita contraseñas ni red.
3. **Solo se construye lo que está en la lista.** Si no está en el spec, no se hace. Nada "por si acaso".
4. **Los colores viven en un solo archivo.** `app/globals.css`. Un color escrito en cualquier otro lado es un error.
5. **Cada quien toca sus archivos.** Si necesitas cambiar uno que no es tuyo, lo avisas, no lo cambias.
6. **Si hay dudas, gana el apéndice.** `REPLICATION-APPENDIX.md` tiene la versión correcta de cada archivo.

Están completas en `.specify/memory/constitution.md`.

Lista final antes de presentar

- ☐ Todos pueden clonar el repo
- ☐ El comando del paso 2 funciona en la máquina de cada uno
- ☐ Todos tienen Claude Code y una API key que funciona
- ☐ Todos leyeron la sección §10
- ☐ **Sergio ya corrió T001 a T009 y están en `main`**
- ☐ **Tomás ya tiene el `.env.local`**
- ☐ **Ya construyeron la app completa una vez y arrancó con `npm run dev`**
- ☐ Cada uno sabe de memoria el comando que le toca correr en vivo

Los tres puntos en negrita son los que no se pueden improvisar. Si llegan a la presentación sin haber construido la app al menos una vez, van a descubrir los problemas en vivo.

Si necesitas cambiar algo del plan

Los archivos de `specs/` mandan. Los editas, los subes, y avisas al equipo.

Lo único que **no** debes hacer es regenerar `tasks.md` con `/speckit-tasks`. Ese archivo dice quién es dueño de cada archivo, y eso es lo que permite que tres personas trabajen a la vez. Si lo regeneras, se pierde. Si necesitas tickets nuevos, escríbelos a mano copiando el formato de los que ya están.